CS5730 Operating Systems: 2021 Midterm Solutions

2021 · CS5730 Operating Systems Midterm (2021/11/30)

Both 2021 PDFs ("Operating System Exam_2021" and "final-2021f") contain this same exam.

Essay Questions: Your answers should be as concise and clear as possible

Q1 (5 pt) Ordinarily the `exec()` system call follows the `fork()`. Explain what would happen if a programmer were to inadvertently place the call to `exec()` before the call to `fork()`. 2021

ANSWER

exec() **replaces the calling process's entire memory image** (code, data, heap, stack) with the program named in its parameters. On success, **it never returns**.

So if exec() is placed before fork(), the new program starts running in place of the original one, under the **same process ID**. The rest of the original code, **including the fork() call, is gone and never executed**. **No child process is created**, and the original program's remaining work is not done.

Only if exec() **fails** (returns −1) does execution continue to the fork().

Q2 (5 pt) Why does the socket communication use the port number instead of process id number to identify server or client process? 2021

ANSWER

- A **PID is assigned dynamically** by the local OS. It changes every time the server is restarted and is **unique only within one host**, so a client on another machine cannot know or rely on it.
- A **port number is fixed and well known**, tied to a **service** rather than to a particular process (e.g. HTTP = 80, SSH = 22; ports below 1024 are reserved). A client can find a service just from IP address and port. A socket is identified by **IP:port**, e.g. 161.25.19.8:1625.
- The process behind a service can change (restart, failover, a pool of worker processes or threads) and one process may offer several services. The port stays the same, and the network stack uses it to deliver packets to the right endpoint.

Q3 (8 pt) What is processor affinity? Explain why it needed to be considered during process scheduling on multiple CPU systems. 2021

ANSWER

Processor affinity: a process has an affinity for the processor it is currently running on. It is kept on (or restricted to) that same CPU rather than migrated to another one.

Why it matters on multiple-CPU systems: while a process runs, its recently used data fills that processor's **cache**. If the scheduler migrates it to another CPU, the old cache contents must be **invalidated** and the new processor's cache must be **repopulated**. That causes many cache misses and memory stalls, which is expensive. Keeping the process on the same CPU reuses the "warm" cache. On NUMA systems it also keeps the process close to its local memory.

Soft affinity: the OS tries to keep the process on the same CPU but may migrate it. **Hard affinity:** the process specifies the set of CPUs it may run on. The scheduler must balance affinity against **load balancing** (push/pull migration), which moves processes from busy to idle CPUs.

Q4 (7 pt) Explain why implementing synchronization primitives by disabling interrupts is not appropriate in a single-processor system if the synchronization primitives are to be used in user-level programs. 2021

ANSWER

Disabling interrupts works on a uniprocessor only for **trusted kernel code**. It is the wrong tool for **user-level** synchronization because:

1. **It is a privileged instruction.** Letting user programs execute it breaks dual-mode protection; the hardware traps if user mode tries.
2. **The OS loses control of the CPU.** While interrupts are off, the **timer interrupt** cannot fire, so no preemption or context switch can happen. A buggy or malicious program could disable interrupts and never re-enable them (e.g. an infinite loop in its critical section). It would then **monopolize the CPU** and hang the whole system.
3. **Other events are delayed or lost.** I/O-completion interrupts and clock ticks are postponed, so devices sit idle, the system clock drifts and response time suffers. User critical sections can be long, which makes this worse.

(It also does not scale: on a multiprocessor it only stops the local CPU.) User programs should instead use OS-provided semaphores/mutexes or atomic hardware instructions such as test_and_set or swap.

Q5 (15 pt) Discuss how the following pairs of scheduling criteria conflict in certain settings. 2021

Q5.1 CPU utilization and response time 2021

ANSWER

CPU utilization vs. response time. CPU utilization is highest when context switches are rare, as with FCFS or a long quantum, because each switch is pure overhead. Good response time for interactive users needs **frequent preemption** (a short RR quantum) so every user gets the CPU quickly. That causes many context switches, so the CPU spends more time switching and less doing useful work, which lowers utilization.

Q5.2 Average turnaround time and maximum waiting time 2021

ANSWER

Average turnaround time vs. maximum waiting time. Average turnaround is minimized by running the **shortest jobs first** (SJF/SRTF). But a long job can be postponed again and again as short jobs keep arriving, so it may **starve**. Its waiting time, the **maximum** waiting time, becomes very large (unbounded). Bounding the maximum wait (e.g. with FCFS or aging) raises the average turnaround.

Q5.3 I/O device utilization and CPU utilization 2021

ANSWER

I/O device utilization vs. CPU utilization. Keeping devices busy means running **I/O-bound** processes as soon as their I/O completes, so they can issue the next request quickly. This means frequently preempting CPU-bound processes, and the context-switch overhead lowers CPU utilization. Maximizing CPU utilization means letting CPU-bound jobs run long without switching, which leaves the I/O devices **idle** while I/O-bound jobs wait for the CPU.

Q5.4 Deadline and fairness 2021

ANSWER

Deadline vs. fairness. To meet deadlines, the scheduler must always give the CPU to the most urgent process (earliest deadline or real-time priority), regardless of how much CPU it has already had. Other processes may wait a long time or **starve**, which is unfair. A **fair** scheduler gives everyone an equal share, so urgent tasks may not get enough CPU in time and **miss their deadlines**.

Q5.5 Throughput and context switch overhead 2021

ANSWER

Throughput vs. context-switch overhead. Throughput (jobs completed per unit time) is improved by finishing short jobs quickly, which needs preemption: SJF/SRTF or a small RR quantum. Every preemption adds **context-switch overhead**, during which no useful work is done. Minimizing switches (FCFS, large quantum) traps short jobs behind long ones (**convoy effect**), so fewer jobs complete per unit time. If the quantum is too small, overhead itself eats the throughput.

6. Multilevel queue scheduling

Q6.1 (5 pt) List two possible scheduling among queues, which we discussed in class. 2021

ANSWER

Two ways to schedule **among** the queues of a multilevel queue:

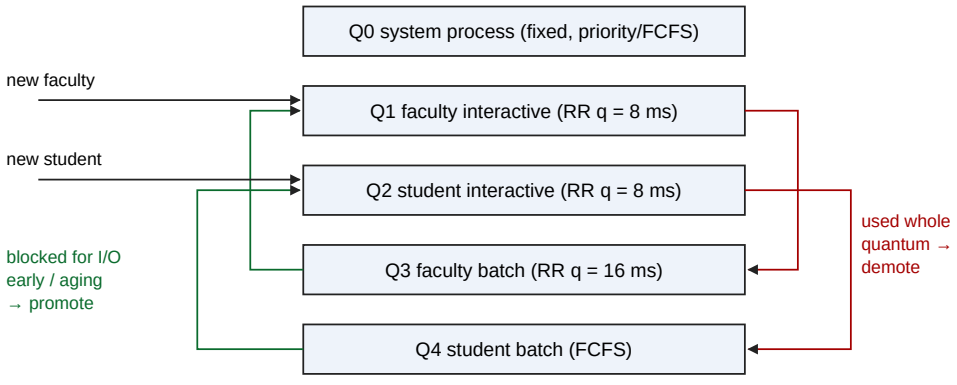
1. **Fixed-priority (preemptive) scheduling.** Serve all processes in a higher-priority queue (e.g. foreground/interactive) before any process in a lower queue (background/batch). A process arriving in a higher queue preempts a lower one. Lower queues may **starve**.
2. **Time slicing among queues.** Each queue gets a fixed share of CPU time, which it schedules among its own processes. Example: 80% to the foreground queue (RR) and 20% to the background queue (FCFS).

Q6.2 (10 pt) Modify the following multilevel queue scheduling system such that a new process will automatically enter the appropriate queue after a few runs. 2021

system process
faculty interactive process
student interactive process
faculty batch process
student batch process

ANSWER

Turn it into a **multilevel feedback queue**: a process may **move between queues** according to its observed behaviour, so it reaches the right queue automatically after a few runs.



Higher queue preempts lower (fixed priority between queues); Q0 never moves.

- A new process enters the **interactive** queue of its class (faculty → Q1, student → Q2), which uses a short quantum.
- If it **uses its whole quantum**, it is CPU-bound, so it is **demoted** to the batch queue of its class (Q1→Q3, Q2→Q4).
- If it **blocks for I/O before the quantum ends**, it is interactive, so it stays in or is **promoted** back to the interactive queue (Q3→Q1, Q4→Q2).
- **Aging** promotes processes that wait too long, preventing starvation. System processes stay fixed in Q0.

Defined by: number of queues, scheduling algorithm per queue, when to promote, when to demote, and which queue a new process enters.

7. Multilevel queue scheduling **(heading as printed; the questions are about synchronization)**

Q7.1 (5 pt) What is **busy-waiting** in the discussion of critical section problem? Explain how **busy-waiting** may degrade the system performance. 2021

ANSWER

Busy waiting (spinning) means a process waiting to enter its critical section stays in a loop that keeps testing a condition until it can enter, e.g. `while (test_and_set(&lock));` or `while (S <= 0);`. It never gives up the CPU while waiting.

Degraded performance: the waiting process **wastes CPU cycles** doing no useful work, and other ready processes could have used them. On a single CPU, the spinner uses its entire time slice while the lock holder cannot run to release the lock, so everyone waits longer. If the spinner has higher priority than the lock holder, the holder may never run, and the system can even **deadlock** (priority inversion). Busy waiting is acceptable only for very short waits on multiprocessors, where it avoids a context switch.

Q7.2 (10 pt) How to implement semaphore without **busy-waiting** using `block` and `wakeup`? You need to write down the definition of data structure `semaphore` and `wait` and `signal` operations. 2021

ANSWER

Each semaphore has an integer value and a **queue of waiting processes**. Instead of spinning, a process that must wait is **blocked** (moved to the waiting state and placed on the semaphore's queue). `signal` **wakes up** one waiting process (moves it to the ready queue).

```
typedef struct {
    int value;
    struct process *list;  /* queue of waiting PCBs */
} semaphore;

void wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();                /* running -> waiting */
    }
}

void signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);              /* waiting -> ready */
    }
}
```

If value is negative, |value| is the number of waiting processes. block() and wakeup() are system calls. wait() and signal() must still be atomic: disable interrupts inside them on a uniprocessor, or use a spinlock on a multiprocessor. That busy wait is limited to these few instructions, so it is short and rare.

Q8 (10 pt) We have discussed the correct solution to the consumer-producer problem, which does not use any synchronization mechanism, as shown below. Please explain why it can only use `BUFFER_SIZE-1` elements of buffer. 2021

```
Producer:
    while (true) {
        /* produce an item in next produced */
        while (((in + 1) % BUFFER_SIZE) == out)
            ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
    }
```

```
Consumer:
    while (true) {
        while (in == out)
            ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        /* consume the item in next consumed */
    }
```

ANSWER

The solution uses only `in` (next free slot) and `out` (next full slot) to describe the buffer state, and it defines **empty** as `in == out` (consumer waits).

If the producer were allowed to fill **all** BUFFER_SIZE slots, then after the last insert `in` would wrap around and also equal `out`. **"Full" would look exactly like "empty"**, and the consumer could not tell them apart. To avoid this ambiguity, the producer treats the buffer as **full when** `(in + 1) % BUFFER_SIZE == out`, so one slot is always left unused.

Example with BUFFER_SIZE = 10: out = 0 and the producer has filled b0–b8, so in = 9. Now (9+1) % 10 = 0 = out, and the producer waits with only **9 items** stored. b9 stays empty; filling it would make in = 0 = out, which means "empty".

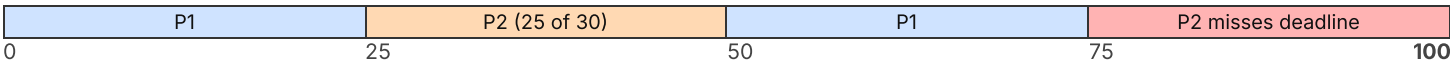
To use all N slots, add a `counter` of items (which then needs synchronization because counter++/counter-- race) or a full/empty flag.

9. Consider two processes, P_1 and P_2, where $p_1 = d_1 = 50$, $t_1 = 25$, $p_2 = d_2 = 75$, and $t_2 = 30$.

Q9.1 (10 pt) Can these two processes be scheduled using rate-monotonic scheduling (RMS)? Illustrate your answer using a Gantt chart such as the ones we did in class. 2021

ANSWER

CPU utilization U = 25/50 + 30/75 = 0.50 + 0.40 = **0.90**. The RMS guaranteed bound for N = 2 is $2(2^{1/2} - 1) \approx$ **0.83**. Since U > 0.83, schedulability is not guaranteed, so draw the chart. RMS uses static priority by period: P1 (period 50) has **higher priority** than P2 (period 75).



- 0–25: P1 runs (deadline 50 met).
- 25–50: P2 runs, but at t = 50 P1's second period starts and, having higher priority, **preempts P2**. P2 has done only 25 of its 30 ms.
- 50–75: P1 runs (deadline 100 met).
- t = 75: P2's deadline arrives with 5 ms still left, so **P2 misses its deadline**.

No, P1 and P2 cannot be scheduled with RMS.

Q9.2 (10 pt) Illustrate the scheduling of these two processes using earliest-deadline-first (EDF) scheduling. 2021

ANSWER

EDF assigns priority dynamically: the job with the **earliest absolute deadline** runs (on a tie, the running job continues). U = 0.90 ≤ 1, so EDF can schedule them.



- t=0: P1 (deadline 50) vs P2 (deadline 75), so P1 runs 0–25. Then P2 starts at 25.
- t=50: P1's 2nd job arrives (deadline 100). P2's deadline 75 is earlier, so **P2 is not preempted** and finishes at 55 (deadline 75 met). P1 runs from 55.
- t=75: P2's 2nd job arrives (deadline 150). P1's deadline 100 is earlier, so P1 finishes at 80 (met). P2 runs 80–110.
- t=100: P1's 3rd job (deadline 150) ties with P2, so no preemption. P2 finishes at 110 (met), and P1 runs 110–135 (met). Idle 135–150.
- At t = 150 (LCM of 50 and 75) both tasks restart, so the pattern repeats.

All deadlines are met under EDF.